

Implementation Plan for Training a Transformer Reasoning Model (TRM) for Text-to-SQL Generation on T4 Infrastructure

I. Architectural Adaptation: Mapping TRM to the Text-to-SQL Domain

The successful application of the Transformer Reasoning Model (TRM) to the text-to-SQL task requires a foundational adaptation of its core architectural principles. The model's innovative recursive, multi-stream processing, originally demonstrated on systematic reasoning tasks like Sudoku, must be re-conceptualized to align with the distinct challenges of natural language understanding and structured query generation.¹ This section details the proposed mapping of the text-to-SQL problem onto the TRM framework, presents a robust strategy for linearizing database schemas, and provides a strategic justification for selecting the most suitable TRM variant for this task, given the computational constraints of an NVIDIA T4 GPU.

A. Re-conceptualizing the Three-Stream Input for Text-to-SQL

The central innovation of the TRM architecture is its departure from single-pass generation. Instead, it employs an iterative process that leverages three distinct information streams: a fixed problem statement (x-stream), an evolving solution (y-stream), and a latent "scratchpad" for intermediate reasoning (z-stream).¹ This paradigm maps with remarkable clarity to the logical steps involved in translating a natural language question into an executable SQL query.

The **x-stream** represents the immutable problem definition. For the text-to-SQL task, it is impossible to generate an accurate query without comprehensive knowledge of both the user's natural language question and the structure of the target database.² Therefore, the x-stream will be formulated as a concatenation of a linearized representation of the database schema and the user's question. This combined input will remain static throughout all recursive steps of the model's forward pass, serving as a constant and complete reference point. The proposed structure is: $x = [NL_Question]$.

The **y-stream** embodies the evolving solution—the SQL query itself. During the training phase, this stream is initialized with the tokenized representation of the ground-truth SQL query, providing a target for the model to learn towards.¹ During inference, the y-stream is iteratively refined. The model's second recursive phase, the `n_refinement_steps`, is dedicated to updating the embeddings within this stream, progressively transforming an initial state into a complete and syntactically correct SQL query.¹ This iterative refinement process is a more sophisticated mechanism than standard token-by-token autoregressive decoding, as it allows the model to revise and improve the entire solution representation at each step.

The **z-stream** functions as the model's latent reasoning space or "scratchpad." As specified in the TRM implementation, this stream is initialized with random noise and receives no direct supervision from the training data.¹ Its purpose is to facilitate the complex intermediate computations required to bridge the gap between the problem statement in the x-stream and the final solution in the y-stream. The first phase of the recursive process, `n_reasoning_steps`, is dedicated exclusively to updating this z-stream, allowing the model to "build understanding" before it begins to formulate an answer.¹

This architectural separation suggests a powerful emergent capability for the text-to-SQL task. The process of generating a complex SQL query is not monolithic; it involves a sequence of logical sub-tasks, most notably schema linking and query planning. Schema linking—the identification of the relevant tables and columns from the database schema that are necessary to answer a given question—is a well-established critical precursor to successful SQL generation.² A logical approach to solving a text-to-SQL problem would first determine *what* data is needed (schema linking) and then determine *how* to retrieve it (query planning and syntax construction). The TRM's two-phase reasoning process appears uniquely suited to mirror this cognitive workflow. It is hypothesized that during the initial `n_reasoning_steps`, the model will learn to use the z-stream to perform these crucial preliminary steps. The embeddings within the z-stream will evolve to represent an abstract query plan: identifying necessary tables, columns for selection, join paths, and filtering conditions. Subsequently, during the `n_refinement_steps`, the model will leverage this fully-formed plan in the z-stream to guide the precise, syntactically correct construction of the final SQL query in the y-stream. This potential for emergent compositional reasoning makes TRM a compelling architecture for this domain.

B. A Strategy for Effective Schema Linearization

Transformer-based models are fundamentally designed to process one-dimensional sequences of tokens.⁶ A core challenge in text-to-SQL is the conversion of the inherently relational and structured database schema into such a linear format. The quality and clarity of this linearization process are paramount, as it directly dictates the model's ability to comprehend the database's structure, including its tables, columns, data types, and the critical relationships defined by primary and foreign keys.³

The standard dataset for this task, Spider, provides schema information in a structured

tables.json file.⁷ This file contains a list of databases, with each entry detailing its db_id, table names, column names (with an associated table index), column types, primary keys, and foreign keys.⁸ An effective linearization strategy must capture all of this information losslessly while providing explicit structural cues that the model can learn to interpret.

To achieve this, a format incorporating a set of custom special tokens is proposed. These tokens will demarcate the different components of the schema, making the structure explicit within the linear string.

The proposed linearization format is as follows:

db_id table_name_1 [COL] col_1_name (col_1_type) [PK][COL] col_2_name (col_2_type)...
table_name_2... [FK] table_a.col_x = table_b.col_y...

The components of this format are defined as:

- ``: A token marking the beginning of the schema definition, followed by the database identifier.
- ``: A token preceding each table definition, followed by the table's name.
- **[COL]**: A token preceding each column, followed by the column's name and its data type enclosed in parentheses (e.g., (text), (number)).
- **[PK]**: A token appended directly after a column's definition to signify that it is a primary key for its table.
- **[FK]**: A token marking the beginning of the foreign key section, which is followed by a list of explicit relationship declarations in the format table_name.column_name = other_table_name.other_column_name. This explicit format is more directly interpretable by the model than the index-based representation in the source JSON.

The following table provides a concrete example of this transformation, demonstrating how a schema from the Spider dataset would be converted into the model-ready linearized string.

Spider tables.json Snippet (for concert_singer db)	Proposed Linearized String
{ "db_id": "concert_singer", "table_names": ["stadium", "singer", "concert", "singer_in_concert"], "column_names": [0, "Location", [0, "Name"], [0, "Capacity"], [0, "Highest", [0, "Lowest", [0, "Average"], [1, "Name"], [1, "Country"], [1, "Age"], [1, "Is_male"], [2, "concert_Name"], [0, "column_types": ["number", "text", "text", "number", "number", "number", "number", "number", "number", "text", "text", "text", "text", "text", "number", "number", "number", "number"], "primary_keys": ["concert_ID"], "foreign_keys": [{"table_a": "concert", "col_a": "Stadium_ID", "table_b": "stadium", "col_b": "Stadium_ID"}, {"table_a": "concert", "col_a": "singer_in_concert", "table_b": "singer", "col_b": "Singer_ID"}] }	concert_singer stadium [COL] Stadium_ID (number) [PK][COL] Location (text) [COL] Name (text) [COL] Capacity (number) [COL] Highest (number) [COL] Lowest (number) [COL] Average (number) singer [COL] Singer_ID (number) [PK][COL] Name (text) [COL] Country (text) [COL] Song_Name (text) [COL] Song_release_year (text) [COL] Age (number) [COL] Is_male (others) concert [COL] concert_ID (number) [PK][COL] concert_Name (text) [COL] Theme (text) [COL] Stadium_ID (text) [COL] Year (number) singer_in_concert [COL] concert_ID (number) [COL] Singer_ID (number) [FK] concert.Stadium_ID = stadium.Stadium_ID singer_in_concert.concert_ID =

	concert.concert_ID singer_in_concert.Singer_ID = singer.Singer_ID
--	--

C. Strategic Model Variant Selection: TRM-MLP as the Primary Candidate

The TRM source code and accompanying explanation detail two architectural variants: TRM-Att, which uses standard multi-head self-attention layers within its transformer blocks, and TRM-MLP, which replaces these attention mechanisms with simpler Multi-Layer Perceptron (MLP) or feed-forward layers.¹ While attention is often considered the cornerstone of modern transformer architectures, the evidence presented suggests that for certain classes of problems, the simpler MLP-based variant is not only sufficient but superior. Specifically, for the highly structured task of solving Sudoku puzzles, the TRM-MLP variant achieves a remarkable 87.4% accuracy, significantly outperforming the TRM-Att variant's 74.7%.¹ The author concludes that for such structured problems, the MLP-only version is "faster, uses less memory, and learns better".¹ This finding has profound implications for the text-to-SQL task.

The generation of SQL queries, much like solving Sudoku, is a problem governed by rigid rules and a strict, unforgiving syntax. While the natural language input is fluid and requires semantic understanding, the output must conform precisely to the grammar of SQL. Multi-head attention is exceptionally powerful at capturing long-range semantic dependencies in free-form text. However, this very flexibility can be a liability when the task demands strict adherence to a formal structure. An MLP-only architecture, by contrast, forces the model to rely more heavily on positional encodings and the fixed, learned transformations within its layers. This architectural constraint may compel the model to learn the syntactic structure of SQL more robustly, as it cannot rely on diffuse semantic similarities captured by an attention mechanism.

Furthermore, the TRM-MLP variant offers significant practical advantages for training on a memory-constrained T4 GPU. It contains approximately 30% fewer parameters than its attention-based counterpart (~5M vs. ~7M).¹ This reduction in model size directly translates to a smaller memory footprint, which can enable the use of larger batch sizes or a higher embedding dimension (`d_model`), both of which can lead to more stable and effective training. Given these factors, the implementation plan will prioritize the **TRM-MLP** variant as the primary candidate for this task. It represents a more computationally efficient and potentially more effective architecture for generating syntactically correct SQL, aligning perfectly with the project's hardware constraints and the structural nature of the problem domain. The TRM-Att variant will be retained as a secondary candidate for comparative experiments, should resources and time permit.

II. Data Engineering and Vocabulary Construction

With the architectural adaptations defined, the next phase focuses on the practical implementation of the data pipeline. This involves selecting and understanding the training dataset, designing a preprocessing workflow to transform the raw data into the model-ready format specified in the previous section, and developing a specialized vocabulary capable of handling the hybrid nature of the text-to-SQL task.

A. The Spider Dataset: A Blueprint for Training Data

The Spider dataset is the established benchmark for developing and evaluating complex, cross-domain text-to-SQL models.⁷ Its design addresses the key limitations of prior datasets by including multiple complex databases, ensuring that the training and test sets do not share database schemas. This forces models to learn to generalize to new, unseen database structures rather than memorizing query templates for a single domain.⁷ The dataset comprises 10,181 natural language questions paired with 5,693 unique, complex SQL queries, spanning 200 different databases across 138 domains.⁷ This scale and diversity make it the ideal resource for training a robust TRM.

The implementation will utilize three key files from the Spider dataset distribution:

1. `train_spider.json` (and `train_others.json`): These files contain the primary training examples. Each entry is a JSON object containing the natural language question, the corresponding ground-truth SQL query, and the `db_id` that links it to a specific database schema.⁷
2. `dev.json`: This file serves as the validation set and follows the same format as the training files. It will be used to monitor model performance during training and for hyperparameter tuning.
3. `tables.json`: This file contains the schema definitions for all 200 databases in the dataset. It is a list of JSON objects, where each object corresponds to a database and is identified by its `db_id`. This file is the source for the schema linearization process.⁸

The structure of these files provides all the necessary components to construct the x-stream (from question and the `tables.json` entry for the `db_id`) and the y-stream (from query) for each training example.

B. The Preprocessing Pipeline: From Raw Data to Model-Ready Tensors

A systematic preprocessing pipeline is required to convert the raw Spider JSON files into tokenized tensors that the TRM can consume. This pipeline will be implemented as a series of

deterministic steps.

- **Step 1: Schema Parsing and Caching.** The tables.json file will be loaded and parsed once at the beginning of the preprocessing script. The information for each database will be extracted and stored in a Python dictionary, keyed by db_id. This creates an efficient, in-memory lookup table for schema information, avoiding redundant file parsing for each training example.
- **Step 2: Schema Linearization.** The script will iterate through each example in the training and validation JSON files (train_spider.json, dev.json). For each (question, query, db_id) triplet, it will use the db_id to retrieve the corresponding schema from the cached dictionary. This schema will then be converted into the linearized string format defined in Section I.B.
- **Step 3: Stream Concatenation.** The resulting linearized schema string will be concatenated with the question string. A special separator token, <SEP>, will be inserted between them to provide a clear boundary for the model. This final concatenated string constitutes the input text for the x-stream. The query string from the dataset entry serves as the target text for the y-stream.
- **Step 4: Tokenization.** The input text (x-stream) and target text (y-stream) for each example will be processed by the custom tokenizer (detailed in Section II.C). This step converts the text strings into sequences of integer token IDs, which are the fundamental input for the model's embedding layer.
- **Step 5: Padding and Batching.** A custom collate_fn will be implemented for use with the PyTorch DataLoader.¹ When creating a batch of examples, this function will dynamically pad the token ID sequences for both the x-stream and y-stream to the maximum length present within that specific batch. This ensures that all tensors within a batch have a uniform shape, a requirement for efficient processing on the GPU. A padding token ID (e.g., 0) will be used, and the loss function will be configured to ignore this token during gradient computation.

C. Vocabulary Development for a Hybrid Language Task

The text-to-SQL task presents a unique vocabulary challenge, as the model must be fluent in three distinct "languages": the natural language of the user's question, the formal syntax of SQL, and the structured "language" of our linearized schema defined by special tokens. A generic, off-the-shelf tokenizer is inadequate for this hybrid domain.

The proposed strategy is to extend a powerful, pre-trained tokenizer rather than training one from scratch. This approach leverages the vast linguistic knowledge already encoded in a large language model's tokenizer while adapting it to our specific needs.

1. **Base Tokenizer Selection:** The tokenizer from a pre-trained sequence-to-sequence model is the ideal starting point. The BART (facebook/bart-large) tokenizer is an excellent candidate.¹³ BART's architecture, consisting of a bidirectional encoder and an autoregressive decoder, is well-suited for conditional generation tasks like text-to-SQL,

and its tokenizer is robust and well-supported.

2. **Vocabulary Extension:** The base tokenizer's vocabulary will be augmented with all necessary domain-specific tokens. This includes a comprehensive list of SQL keywords (e.g., SELECT, FROM, WHERE, JOIN, GROUP BY, ORDER BY, LIMIT, DISTINCT, AVG, COUNT, MAX, MIN, SUM, INTERSECT, EXCEPT, UNION) and the custom special tokens defined for schema linearization (, , [COL], [PK], [FK], <SEP>). This can be accomplished using the `add_tokens` method in the Hugging Face transformers library.
3. **Resizing Model Embeddings:** After new tokens are added to the tokenizer, its vocabulary size increases. The model's token embedding layer (`nn.Embedding`) must be correspondingly resized to accommodate these new tokens. The weights for the original vocabulary tokens are preserved, while the embeddings for the newly added tokens are randomly initialized. These new embeddings will be learned and fine-tuned alongside the rest of the model during the training process.¹⁸

This approach of creating a single, unified vocabulary is not merely a technical necessity but a strategic choice to enhance model performance. By ensuring that a table name like `singer` is represented by the exact same token(s) whether it appears in the schema, the question, or the target SQL, we strongly ground the model's generation process. This reduces the ambiguity that can lead to common text-to-SQL failure modes, such as the "hallucination" of incorrect table or column names. It effectively forces the model to learn a copy-like mechanism, ensuring that the entities it uses in the generated SQL are drawn directly from the context provided in the x-stream, thereby improving the overall reliability and accuracy of the system.

III. T4-Optimized Training Regimen

Training the TRM, a model with a deep, recursive forward pass, on a single NVIDIA T4 GPU presents significant computational and memory challenges. This section outlines a training regimen specifically designed to address these constraints. It details the mandatory use of automatic mixed precision and gradient checkpointing, establishes a conservative baseline for hyperparameters, and incorporates essential techniques for ensuring stable convergence.

A. Maximizing T4 Performance with Automatic Mixed Precision (AMP) and Gradient Checkpointing

The NVIDIA T4 GPU is equipped with Tensor Cores, specialized hardware units that deliver substantial performance gains for matrix multiplication and convolution operations when using lower-precision floating-point formats like FP16.¹⁹ However, its 16 GB of VRAM imposes a strict limit on model size and batch size. To train the TRM effectively within these constraints, a multi-pronged optimization strategy is required.

Automatic Mixed Precision (AMP) is a non-negotiable component of this training plan. AMP

intelligently combines the use of 32-bit floating-point (FP32) and 16-bit floating-point (FP16) arithmetic. It reduces the memory footprint of model weights, gradients, and optimizer states by nearly half and can accelerate training by a factor of 2-3x on compatible hardware.²¹ This is crucial for fitting a sufficiently large batch into the T4's memory. The implementation will leverage PyTorch's native AMP support (torch.cuda.amp):

- **autocast:** The model's entire forward pass, including the recursive reasoning loop, will be executed within a with autocast(): context manager. This instructs PyTorch to automatically cast operations to FP16 where it is safe and beneficial (e.g., linear layers in the MLP blocks) while maintaining FP32 for operations that are numerically sensitive (e.g., loss calculations, softmax).¹
- **GradScaler:** A torch.cuda.amp.GradScaler object will be instantiated to manage the backward pass. Gradients computed in FP16 have a smaller dynamic range and are susceptible to "underflow"—becoming zero and halting learning. The GradScaler mitigates this by scaling the loss value up before the backward pass, which inflates the gradients into a safe range. It then automatically unscales these gradients before the optimizer updates the weights, ensuring the update magnitude is correct.¹

While AMP addresses the memory size of individual tensors, the TRM's recursive nature introduces a more severe challenge: the memory cost of the computation graph itself. A standard backward pass requires storing the intermediate activations for every operation in the forward pass. For the TRM, which performs $n_{\text{reasoning_steps}} + n_{\text{refinement_steps}}$ (e.g., 24) internal forward passes for every single training example, the cumulative memory required to store these activations would be immense, far exceeding the T4's capacity even with a batch size of one.¹

The solution to this is **gradient checkpointing**. This technique modifies the backward pass to avoid storing most intermediate activations. Instead of retrieving them from memory, it recomputes them on-the-fly as they are needed for gradient calculation. This represents a classic trade-off: it increases the computational cost of the backward pass but provides a dramatic reduction in memory usage.¹ For a deeply recursive model like TRM on a memory-constrained GPU, this trade-off is essential for feasibility. The implementation will use torch.utils.checkpoint.checkpoint to wrap the forward method of each TransformerBlock. This is the single most critical adaptation to make training viable on the target hardware.

B. Establishing a Baseline Hyperparameter Configuration

The selection of initial hyperparameters is guided by the parameters used in the original TRM implementation, with conservative adjustments to respect the T4's limitations. The following table provides a robust starting point for training and subsequent tuning.

Hyperparameter	Recommended Value	Justification
d_model	256	A modest embedding dimension that balances model

		capacity with memory usage. This value is consistent with the example provided in the TRM source code. ¹
n_layers	4	The number of transformer blocks used in the original paper's configuration, providing sufficient representational depth for the task. ¹
d_ff	1024 ($4 \times d_{\text{model}}$)	The standard expansion factor for the feed-forward network dimension in transformer architectures, as used in the TRM implementation. ¹
n_reasoning_steps	8	The value specified in the TRM paper and code. This serves as a solid baseline before exploring the impact of more or fewer reasoning steps. ¹
n_refinement_steps	16	The value specified in the TRM paper and code. This provides a substantial number of iterations for the model to refine its output SQL query. ¹
latent_len	32	The default length of the z-stream (reasoning scratchpad) used in the TRM code examples. This provides a reasonably sized latent space for planning. ¹
batch_size	8–16	A conservative batch size range. The final value will be determined empirically by profiling memory usage to maximize VRAM utilization without encountering out-of-memory (OOM) errors. This is highly dependent on the maximum sequence length after tokenization.
learning_rate	1×10^{-4}	A standard and effective

		learning rate for the Adam optimizer when training transformer models, as used in the TRM training script. ¹
optimizer	Adam	The Adam optimizer is a robust choice for training deep neural networks and is the one used in the provided TRM training example. ¹
warmup_steps	1000	As recommended in the "Essential Training Tricks" section of the TRM guide, a warmup period prevents large, destabilizing weight updates at the beginning of training. ¹
gradient_clip_norm	1.0	A critical parameter to prevent exploding gradients, which are a significant risk in a deeply recursive model. This value is recommended in the TRM guide. ¹

C. Essential Training Enhancements: Gradient Clipping and Learning Rate Scheduling

Beyond the core optimizations, two additional techniques highlighted in the TRM guide are essential for ensuring a stable and successful training process.

Gradient Clipping: The recursive application of transformer blocks makes the effective depth of the TRM's computation graph exceptionally large. This creates a high risk of the exploding gradients problem, where gradients grow exponentially during the backward pass, leading to NaN values and a complete collapse of the training process. To prevent this, gradient clipping will be applied at every training step. Specifically, `torch.nn.utils.clip_grad_norm_` will be called after the `scaler.scale(loss).backward()` call and before `scaler.step(optimizer)`. This function rescales the gradients if their L2 norm exceeds a specified threshold (1.0), effectively capping their maximum magnitude and ensuring numerical stability.¹

Learning Rate Scheduling: A static learning rate is often suboptimal for training large models. A dynamic learning rate schedule, particularly one that incorporates a warmup phase, leads to more reliable convergence. The implementation will use a scheduler that combines a linear warmup for the first 1000 steps with a subsequent cosine decay for the remainder of training, as detailed in the TRM guide's `get_lr_scheduler` function.¹ The initial warmup phase

gradually increases the learning rate from zero to its target value, allowing the model to adapt gently from its random initialization. The cosine decay phase then smoothly anneals the learning rate, which helps the model to settle into a broad, stable minimum in the loss landscape.

IV. A Framework for Rigorous Evaluation and Iteration

Evaluating the performance of a text-to-SQL model requires more than just calculating loss on a validation set. A robust framework must assess the functional correctness of the generated queries and provide methods for introspective analysis to understand the model's behavior. This section outlines the core metrics, a plan for model analysis, and a roadmap for systematic improvement.

A. Core Metrics: Execution Accuracy and Exact Match

The evaluation of the trained TRM will be based on two standard and complementary metrics in the text-to-SQL literature.

The primary and most important metric is **Execution Accuracy (EX)**. This metric directly measures the functional correctness of the generated SQL. The process involves executing the model-generated query against the corresponding database from the Spider dataset and comparing the returned result set with the results from executing the ground-truth query.²⁵ A query is considered correct if its result set is identical to the ground truth, ignoring row and column order. EX is the ultimate measure of a model's utility, as it confirms whether the generated query actually solves the user's request.

The secondary metric is **Exact Set Match (EM)**. This metric provides a more granular assessment of syntactic and structural correctness without requiring database execution. It works by decomposing both the predicted and ground-truth SQL queries into their constituent clauses (e.g., SELECT, FROM, WHERE, GROUP BY). It then performs a set-based comparison of the components within each clause, making it robust to inconsequential variations in ordering (e.g., the order of columns in a SELECT statement or conditions in a WHERE clause).²⁶ EM is a valuable proxy for correctness that can be computed quickly, making it useful for rapid validation during development cycles.

An evaluation harness will be developed to automate this process. For each example in the dev set, the script will:

1. Feed the linearized schema and question to the trained model to generate a SQL query string.
2. Parse both the predicted and gold queries to calculate the Exact Set Match score.
3. Establish a connection to the appropriate SQLite database provided with the Spider dataset.
4. Execute the gold query to obtain the ground-truth result set.

5. Execute the predicted query within a try-except block to handle potential syntax errors. If an error occurs, the query is marked as incorrect for EX.
6. If both queries execute successfully, their result sets are compared to determine the Execution Accuracy score.

B. Introspective Analysis: Visualizing the Reasoning Stream

A unique advantage of the TRM architecture is the ability to inspect its intermediate reasoning process. The `recursive_reasoning` function can be modified to return the entire trajectory of the z-stream and y-stream states across all recursive steps.¹ This capability allows for a qualitative analysis that goes beyond simple performance metrics, offering a window into *how* the model arrives at its solution.

To validate the hypothesis that the z-stream learns to function as an emergent schema linker and query planner, the following analysis will be performed:

1. For a selection of examples from the validation set, the model will be run in evaluation mode, and the sequence of `z_states` (one for each of the `n_reasoning_steps`) will be saved.
2. The token embeddings for all table and column names present in the input linearized schema for that example will be extracted from the model's embedding layer.
3. For each reasoning step `i` from 1 to `n_reasoning_steps`, the cosine similarity will be calculated between the embeddings in the `z_state_i` and the embeddings of each schema component (table and column names).
4. These similarity scores will be plotted over the reasoning steps, tracking how the z-stream's focus evolves.

If the hypothesis is correct, this visualization should reveal a distinct pattern. In the initial reasoning steps, the average similarity between the z-stream embeddings and the embeddings of the *relevant* schema components (those required to answer the question) should steadily increase, while similarity to irrelevant components should remain low. This would provide strong evidence that the model is learning the compositional strategy of first identifying the necessary data sources (schema linking) and encoding this plan into its latent z-stream before proceeding to the refinement phase. Such a finding would not only validate the choice of the TRM architecture for this task but also offer profound insights into the mechanisms of emergent reasoning in transformer models.

C. A Roadmap for Iterative Improvement

The initial training and evaluation will establish a performance baseline. Subsequent efforts will focus on systematic experimentation and analysis to improve upon this baseline. The following roadmap outlines key areas for iterative improvement.

- **Step 1: Tuning the Reasoning-Refinement Balance.** The ratio of `n_reasoning_steps` to

n_refinement_steps is a crucial hyperparameter unique to the TRM. Experiments will be conducted by varying this balance (e.g., increasing reasoning steps to 12 and decreasing refinement to 12, or vice versa) to determine if performance on the text-to-SQL task benefits more from extended "thinking" (planning in the z-stream) or extended "writing" (refining the y-stream).

- **Step 2: Experimentation with Schema Linearization.** While the proposed linearization format is comprehensive, alternative representations could yield better performance. Future iterations could explore variations, such as including a few sample data values for each column (e.g., [COL] Country (text) {USA, France,...}) to provide the model with more semantic context, or using different structural delimiters.
- **Step 3: Comparative Analysis of TRM-MLP vs. TRM-Att.** Once a stable training pipeline and a strong baseline are established with the prioritized TRM-MLP variant, a direct, controlled comparison against the TRM-Att variant will be conducted. This A/B test will provide definitive empirical evidence regarding the suitability of MLP-based versus attention-based recursive architectures for this structured generation task, testing the hypothesis outlined in Section I.C.
- **Step 4: Qualitative Error Analysis.** A crucial step for targeted improvement is a deep, qualitative analysis of the model's failures on the validation set. Errors will be categorized into distinct classes, such as:
 - **Syntax Errors:** Generating grammatically incorrect SQL.
 - **Schema Grounding Errors:** Hallucinating non-existent table or column names.
 - **Join Errors:** Selecting incorrect columns for join conditions or failing to join necessary tables.
 - **Logic Errors:** Misinterpreting conditions in the WHERE clause (e.g., using > instead of <) or incorrect aggregation logic.By identifying the most frequent error categories, future efforts can be focused on the model's specific weaknesses, whether through targeted data augmentation, adjustments to the schema representation, or architectural modifications.

Conclusion

This report has detailed a comprehensive and actionable plan for adapting and training the Transformer Reasoning Model for the text-to-SQL generation task, specifically tailored for execution on a single NVIDIA T4 GPU. The proposed strategy is grounded in a thorough analysis of the TRM architecture and the specific demands of the text-to-SQL domain. The viability of this plan hinges on several key strategic decisions. First, the re-conceptualization of the TRM's three-stream input provides a natural mapping for the text-to-SQL problem, positing that the model's latent z-stream can learn to perform the critical sub-tasks of schema linking and query planning. Second, the prioritization of the TRM-MLP variant offers a path that is not only more computationally efficient and suitable for

the constrained hardware but is also potentially more effective for a task requiring rigid syntactic correctness.

Crucially, the plan addresses the formidable memory challenges of training a deeply recursive model on a T4 GPU by mandating the combined use of Automatic Mixed Precision and gradient checkpointing. These are not optional optimizations but essential enablers for the project's feasibility. The provided baseline hyperparameters and training enhancements, such as gradient clipping and learning rate scheduling, establish a robust foundation for achieving stable convergence.

Finally, the proposed evaluation framework, centered on Execution Accuracy and supplemented by introspective analysis of the model's reasoning process, provides a clear methodology for measuring success and gaining deeper scientific understanding. By following this implementation plan, it is possible to rigorously train and evaluate the TRM, exploring the potential of recursive reasoning architectures to advance the state of the art in complex, structured language tasks.

Works cited

1. Let's Build a Tiny Recursive Model from Scratch with Complete Code- When Tiny Ne.pdf
2. Enhancing Text-to-SQL with Open-source LLMs - OpenReview, accessed on October 18, 2025, <https://openreview.net/pdf?id=jkYV7OGNol>
3. X-SQL: Expert Schema Linking and Understanding of Text-to-SQL with Multi-LLMs - arXiv, accessed on October 18, 2025, <https://arxiv.org/html/2509.05899v1>
4. Analyzing the Effectiveness of Large Language Models on Text-to-SQL Synthesis - arXiv, accessed on October 18, 2025, <https://arxiv.org/html/2401.12379v1>
5. RSL-SQL: Robust Schema Linking in Text-to-SQL Generation - arXiv, accessed on October 18, 2025, <https://arxiv.org/html/2411.00073v2>
6. On Linearizing Structured Data in Encoder-Decoder Language Models: Insights from Text-to-SQL - ACL Anthology, accessed on October 18, 2025, <https://aclanthology.org/2024.naacl-long.8/>
7. Yale University's Spider 1.0 NLP Dataset - Kaggle, accessed on October 18, 2025, <https://www.kaggle.com/datasets/jeromeblanchet/yale-universitys-spider-10-nlp-dataset>
8. taoyds/spider: scripts and baselines for Spider: Yale complex and cross-domain semantic parsing and text-to-SQL challenge - GitHub, accessed on October 18, 2025, <https://github.com/taoyds/spider>
9. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task - ACL Anthology, accessed on October 18, 2025, <https://aclanthology.org/D18-1425.pdf>
10. Spider: Yale Semantic Parsing and Text-to-SQL Challenge, accessed on October 18, 2025, <https://yale-lily.github.io/spider>
11. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task - arXiv, accessed on October 18, 2025,

- <https://arxiv.org/abs/1809.08887>
12. sasmohaimeed/Ar-Spider - GitHub, accessed on October 18, 2025, <https://github.com/sasmohaimeed/Ar-Spider>
 13. Bart Large Nl2sql · Models - Dataloop, accessed on October 18, 2025, https://dataloop.ai/library/model/swastikm_bart-large-nl2sql/
 14. BART - NLP to SQL - Kaggle, accessed on October 18, 2025, <https://www.kaggle.com/code/sbadri/bart-nlp-to-sql>
 15. BART - Hugging Face, accessed on October 18, 2025, https://huggingface.co/docs/transformers/model_doc/bart
 16. Transformers BART Model Explained for Text Summarization - ProjectPro, accessed on October 18, 2025, <https://www.projectpro.io/article/transformers-bart-model-explained/553>
 17. A Novel Approach for text-to-SQL: Dual Transformers Approach | by Shahrukh khan, accessed on October 18, 2025, <https://medium.com/@shahrukhx01/a-novel-approach-for-text-to-sql-dual-transformers-approach-e2a285dfb630>
 18. LLM Fine Tuning Mistral-7B for Text to SQL - Kaggle, accessed on October 18, 2025, <https://www.kaggle.com/code/shorya22/llm-fine-tuning-mistral-7b-for-text-to-sql>
 19. Train With Mixed Precision - NVIDIA Docs, accessed on October 18, 2025, <https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html>
 20. Introducing native PyTorch automatic mixed precision for faster training on NVIDIA GPUs, accessed on October 18, 2025, <https://pytorch.org/blog/accelerating-training-on-nvidia-gpus-with-pytorch-automatic-mixed-precision/>
 21. yasufumy/spider-schema-linking-dataset - GitHub, accessed on October 18, 2025, <https://github.com/yasufumy/spider-schema-linking-dataset>
 22. Mixed Precision — PyTorch Training Performance Guide - GitHub Pages, accessed on October 18, 2025, <https://residentmario.github.io/pytorch-training-performance-guide/mixed-precision.html>
 23. Benchmarking GPUs for Mixed Precision Training with Deep Learning - Paperspace Blog, accessed on October 18, 2025, <https://blog.paperspace.com/mixed-precision-training-benchmark/>
 24. How to implement mixed precision training with PyTorch? - Tencent Cloud, accessed on October 18, 2025, <https://www.tencentcloud.com/techpedia/126165>
 25. Evaluate a Text-to-SQL Agent - Ragas, accessed on October 18, 2025, <https://docs.ragas.io/en/stable/howtos/applications/text2sql/>
 26. Benchmarking the Text-to-SQL Capability of Large Language Models: A Comprehensive Evaluation - arXiv, accessed on October 18, 2025, <https://arxiv.org/html/2403.02951v1>